Trabajo Práctico 2 – Sistemas de Computación

Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

0.1 **Introducción** 4

0.2 **Descripción General del Proyecto** 5

0.3 **Tecnologías Utilizadas** 6

0.4 **Variante without_docker** 7

0.4.1 Flujo de trabajo 7

0.4.2 Componentes principales 7

0.5 **Variante docker_test** 8

0.5.1 Archivo api.py 8

0.5.2 Obtención de datos 8

0.5.3 Modelo de datos 9

0.5.4 Biblioteca compartida y ctypes 9

0.5.5 Transformación C y Assembly 9

0.5.6 Postprocesamiento y visualización 10

0.5.7 Endpoint HTTP 10

0.6 **Variante c_program** 11

0.6.1 Objetivo de la variante 11

0.6.2 Organización de archivos 11

0.6.3 Consulta HTTPS desde C 12

0.6.4 Parseo y filtrado del JSON 12

0.6.5 Transformación y salida final 13

0.6.6 Compilación y depuración 13

0.7 **Análisis de Memoria con GDB** 15

0.7.1 Secuencia 1: estado previo al análisis 15

0.7.2 Secuencia 2: estado durante la ejecución de parse_world_bank_json 18

0.7.3 Secuencia 3: estado posterior al retorno desde la función 20

0.7.4 Interpretación general de la variación del stack 22

0.1 Introducción

El presente trabajo práctico se centra en el estudio y la verificación de las convenciones de llamada entre distintos lenguajes y niveles de abstracción. El objetivo principal del proyecto no es solamente procesar un conjunto de datos, sino comprobar en la práctica cómo deben intercambiarse argumentos, valores de retorno y estructuras de memoria cuando intervienen componentes escritos en Python, C y lenguaje ensamblador.about:blank#blocked

Para llevar adelante esta comprobación, el proyecto utiliza un conjunto de herramientas que permiten poner a prueba dicha convención de llamada en un escenario real. Python se emplea como lenguaje de orquestación y como punto de entrada del sistema; C funciona como capa intermedia para definir una interfaz estable con la rutina nativa; y Assembly implementa la lógica de conversión numérica. A su vez, el uso de `ctypes`, bibliotecas compartidas, scripts de compilación y contenedores Docker permite verificar que la comunicación entre capas se produzca correctamente y de forma reproducible.

Con el fin de contar con datos reales sobre los cuales ejecutar estas pruebas, se toma como fuente la API pública del Banco Mundial y se trabaja con valores del índice de Gini. De este modo, el proyecto combina un caso concreto de procesamiento de datos con un objetivo técnico más profundo: validar que las convenciones de llamada se respeten correctamente a lo largo de todo el flujo, desde la obtención de la información hasta la ejecución del código nativo y la recuperación de los resultados en Python.

0.2 Descripción General del Proyecto

En este proyecto se pueden encontrar tres variantes del trabajo. La primera, `without_docker`, representa una aproximación inicial en la que Python obtiene los datos y delega la conversión a una biblioteca de 32 bits utilizando `msl.loadlib`. La segunda, `docker_test`, constituye una versión más integrada del flujo completo, donde la compilación del código nativo y la ejecución del pipeline se realizan dentro de un contenedor Docker. Y por ultimo, la variante `c_program` se orienta a la observación del comportamiento interno del programa desde GDB, concentrando toda la lógica en un único binario escrito en C.

Desde el punto de vista funcional, ambas variantes comparten la misma idea general: partir de un conjunto de valores del índice de Gini, convertirlos de `float` a `int` mediante código nativo y presentar el resultado final de forma procesable desde Python. La versión con Docker amplía este flujo con la generación de gráficos y con una API HTTP sencilla construida con Flask.

0.3 Tecnologías Utilizadas

El proyecto combina herramientas de distinto nivel, cada una con una responsabilidad bien definida:

Tecnología	**Rol dentro del proyecto**
Python 3	Orquestación general, consumo de API, generación de gráficos y exposición HTTP.
C	Capa puente entre Python y la rutina de bajo nivel.
NASM Assembly	Conversión numérica de punto flotante a entero.
Flask	Exposición de un endpoint simple para devolver datos procesables.
Matplotlib	Generación del gráfico final en formato PNG.
Docker / Docker Compose	Reproducibilidad del entorno de compilación y ejecución.
Banco Mundial API	Fuente externa de datos del índice de Gini.

0.1

La elección de estas herramientas refleja un enfoque de integración multicapa. Python resuelve la lógica de alto nivel con rapidez de desarrollo, mientras que C y Assembly se utilizan para abordar la transformación numérica desde un enfoque más cercano al hardware y a las convenciones de ejecución.

0.4 Variante `without_docker`

La variante `without_docker` funciona como una primera implementación del problema. En esta versión, el archivo `main.py` consulta la API del Banco Mundial para obtener los valores del indicador `SI.POV.GINI` entre 2011 y 2020 y luego usa un cliente de `msl.loadlib` para enviar la lista de flotantes hacia un proceso de 32 bits.

0.4.1 Flujo de trabajo

El flujo de esta variante puede describirse de la siguiente manera:

1. Python consulta la API del Banco Mundial.
2. Se filtran registros por código ISO del país.
3. El cliente `Client64` invoca al servidor `Server32`.
4. El servidor arma arreglos de `ctypes` en memoria de 32 bits.
5. La biblioteca `libbridge.so` procesa cada valor usando C y Assembly.
6. Los resultados enteros vuelven al proceso Python como una lista.

0.4.2 Componentes principales

`main.py` implementa la consulta externa, el filtrado y la prueba de extremo a extremo. La función `obtener_gini_pais` construye una URL de la API, descarga la respuesta JSON y extrae los valores válidos para el país solicitado.

`client.py` define la clase `ClienteBridge`, que hereda de `Client64` y utiliza el método `request32` para comunicarse con el servidor de 32 bits.

`server.py` define la clase `Server`, heredada de `Server32`. Allí se carga la biblioteca compartida, se especifica la firma de la función nativa y se transforman las listas Python en arreglos de `ctypes`.

`bridge.c` contiene la función `bridge`, encargada de recorrer un arreglo de `float` y aplicar la función `convert` sobre cada elemento, dejando el resultado en un arreglo de enteros.

`convert.asm` implementa la conversión de un `float` a `int` mediante la FPU, y posteriormente incrementa el resultado en una unidad. Esto significa que en esta primera versión la lógica de `+1` forma parte del código Assembly.

0.5 Variante `docker_test`

La variante `docker_test` representa una version automatizada del proyecto. Aquí se encuentra un pipeline completo donde Python obtiene los datos, delega la conversión a una biblioteca nativa compilada dentro del contenedor, ajusta los resultados y genera una visualización final.

0.5.1 Archivo `api.py`

El archivo `api.py` concentra la mayor parte de la lógica del sistema. Su estructura interna puede separarse en varios bloques funcionales:

Bloque	**Función principal**
Configuración	Define indicador, país, años, rutas y timeout.
Acceso a API	Construye URLs, realiza la consulta y obtiene registros JSON.
Filtrado	Selecciona registros válidos para un país determinado.
Interoperabilidad nativa	Carga la biblioteca compartida y define `argtypes` y `restype`.
Transformación	Convierte arreglos de `float` a `int` mediante C/ASM.
Postprocesamiento	Incrementa los valores convertidos en `+1`.
Visualización	Genera y guarda un gráfico con Matplotlib.
Exposición HTTP	Devuelve datos de Argentina en un endpoint Flask.

0.2

Esta centralización de responsabilidades facilita seguir el flujo completo del pipeline, aunque también implica que gran parte de la lógica está reunida en un mismo archivo. Para un proyecto académico esto resulta aceptable, ya que permite comprender de manera directa la secuencia de pasos involucrados.

0.5.2 Obtención de datos

La URL base utilizada responde a la forma:

`https://api.worldbank.org/v2/en/country/{country_scope}/indicator/{indicator}`

El indicador seleccionado es `SI.POV.GINI`, que corresponde al índice de Gini. Los parámetros construidos incluyen:

- `format=json`
- `date=2011:2020`
- `page=1`
- `per_page=32500`

La cantidad elevada de elementos por página busca evitar problemas de paginación y asegurar que todos los registros relevantes sean descargados en una sola consulta. Una vez obtenida la respuesta, el programa verifica que el payload tenga la estructura esperada y luego recorre los registros para quedarse únicamente con aquellos que pertenecen al país solicitado y poseen un valor numérico no nulo.

0.5.3 Modelo de datos

La clase `IndicatorPoint`, declarada como `dataclass`, encapsula dos atributos: el año y el valor del indicador. Esta decisión mejora la legibilidad del código porque evita manipular directamente diccionarios crudos a lo largo de todo el pipeline y permite ordenar los resultados de forma clara por año.

0.5.4 Biblioteca compartida y `ctypes`

Para la interoperabilidad con código nativo, `api.py` utiliza `ctypes.CDLL` para cargar el archivo `libfloat_to_int.so`. Una vez abierta la biblioteca, se especifica la firma de la función exportada `convert_float_array_to_int(const float*, int*, int)`.

El uso de `argtypes` y `restype` es importante porque le indica a Python cómo debe pasar los argumentos a la función nativa y cómo interpretar su valor de retorno. Esto reduce errores de interoperabilidad y deja explícito el contrato entre ambos mundos.

0.5.5 Transformación C y Assembly

El archivo `interface_w_asm.c` implementa la función intermedia `convert_float_array_to_int`. Su responsabilidad principal es validar que los punteros de entrada y salida no sean nulos y que la cantidad de elementos no sea negativa. Si la

validación es correcta, delega el trabajo real a la rutina Assembly `asm_convert_float_array_to_int`.

Por su parte, `float_to_int.asm` implementa la lógica de recorrido del arreglo. La rutina recibe:

- `rdi`: puntero al arreglo de `float`
- `rsi`: puntero al arreglo de salida `int`
- `edx`: cantidad de elementos

Luego, para cada posición:

1. Carga el valor flotante en `xmm0` mediante `movss`.
2. Convierte el escalar a entero con `cvttss2si`.
3. Guarda el resultado en el arreglo de salida.
4. Incrementa el índice y repite hasta completar el arreglo.

La instrucción `cvttss2si` realiza una conversión con truncamiento hacia cero. Esto significa que los decimales se descartan, y que la suma posterior de `+1` no se realiza en Assembly en esta versión, sino en Python mediante la función `increment_values`.

0.5.6 Postprocesamiento y visualización

Una vez obtenidos los enteros convertidos, el programa aplica una corrección simple sumando una unidad a cada valor. Esta decisión desacopla la conversión numérica de la regla de negocio adicional, dejando más claro qué parte del pipeline corresponde a bajo nivel y cuál pertenece a la lógica de aplicación.

Después del ajuste, se genera un gráfico con Matplotlib. El archivo de salida se guarda dentro de `runtime/logs` con un nombre que incluye fecha y hora. Esto permite conservar evidencia de distintas ejecuciones y comparar resultados a lo largo del tiempo.

0.5.7 Endpoint HTTP

Además del pipeline principal, `api.py` declara un endpoint Flask en `/api/gini/argentina`. Este endpoint devuelve un arreglo JSON con los valores del indicador correspondientes a Argentina, sin aplicar necesariamente toda la etapa de transformación ni la generación del gráfico. Su inclusión es útil como extensión del trabajo, ya que muestra cómo el proyecto también puede exponer datos a otros consumidores.

0.6 Variante `c_program`

Además de las dos variantes principales del proyecto, se desarrolló una carpeta adicional llamada `c_program`, orientada específicamente a la observación del comportamiento interno del programa desde GDB. A diferencia de `docker_test`, en esta versión no existe una capa de orquestación en Python durante la ejecución normal, sino que todo el flujo principal se resuelve dentro de un único programa escrito en C.

La motivación de esta variante no es reemplazar el pipeline integrado con Python, sino ofrecer una versión más controlada y directa del problema para analizar stack frames, variables locales, buffers y estructuras de memoria en tiempo de ejecución. Esto resulta especialmente útil desde el punto de vista de Sistemas de Computación, porque permite observar con mayor claridad el paso de argumentos, la disposición de datos en memoria y la evolución del estado interno del programa a lo largo de distintas funciones.

0.6.1 Objetivo de la variante

La variante `c_program` busca reproducir la misma idea general del trabajo, pero concentrando todo el procesamiento en C. El flujo implementado puede resumirse del siguiente modo:

1. Construir desde C una consulta hacia la API del Banco Mundial.
2. Establecer una conexión HTTPS con el servidor remoto.
3. Descargar la respuesta HTTP completa.
4. Separar manualmente los headers del cuerpo JSON.
5. Filtrar los registros del país solicitado.
6. Convertir cada valor `float` a `int`.
7. Sumar una unidad a cada valor convertido.
8. Mostrar una tabla y un gráfico ASCII por consola.

De esta manera, la lógica de negocio y la lógica de comunicación con el servicio externo quedan integradas dentro del mismo binario, lo que facilita el seguimiento paso a paso mediante depuración.

0.6.2 Organización de archivos

La carpeta `c_program` contiene una estructura más reducida que la de `docker_test`, compuesta por tres elementos principales:

Archivo	Responsabilidad principal
`main.c`	Implementa toda la lógica del flujo en C.
`Makefile`	Compila el programa y genera el ejecutable en `build/`.
`README.md`	Documenta la ejecución, pruebas de uso y ejemplos de GDB.

0.3

El ejecutable resultante se almacena en la carpeta `build` bajo el nombre `gini_client`, permitiendo separar el código fuente del binario compilado.

0.6.3 Consulta HTTPS desde C

Uno de los aspectos más relevantes de esta variante es que la consulta a la API externa se realiza directamente desde C. Para ello, el programa utiliza una combinación de sockets POSIX y OpenSSL.

La función `connect_tcp_socket` resuelve la dirección del host y establece la conexión TCP base. Luego, `open_tls_connection` construye el contexto TLS, realiza el handshake criptográfico y verifica el certificado del servidor. Una vez establecida la conexión segura, `fetch_https_response` envía una petición HTTP `GET` al endpoint del Banco Mundial y almacena la respuesta completa en un buffer dinámico reservado en memoria.

Este diseño hace explícito un conjunto de mecanismos que en Python suelen estar ocultos detrás de bibliotecas de alto nivel. En consecuencia, el alumno puede observar con más detalle qué ocurre entre la construcción de la consulta y la recepción del payload remoto.

0.6.4 Parseo y filtrado del JSON

Tras recibir la respuesta HTTP, el programa separa la cabecera del cuerpo con la función `extract_http_body`. Luego, la función `parse_world_bank_json` recorre manualmente el contenido JSON utilizando búsquedas sobre cadenas y conversiones con funciones estándar como `strtol` y `strtof`.

Si bien esta estrategia no pretende ser la solución más robusta posible para procesamiento general de JSON, resulta adecuada para un trabajo académico porque vuelve visible el tratamiento explícito de la memoria y de la representación textual de los datos. Cada coincidencia válida se copia en una estructura `IndicatorPoint`, que conserva:

- el año del registro;
- el valor original en punto flotante;
- el valor convertido a entero;
- el valor final ajustado con `+1`.

Posteriormente, `sort_points_by_year` ordena los puntos obtenidos para presentarlos en forma cronológica.

0.6.5 Transformación y salida final

Una vez construidos los registros válidos, la función `convert_points_to_int` trunca cada valor flotante a entero. Luego, `increment_points` aplica el ajuste de `+1` sobre cada elemento convertido. En esta variante, ambas operaciones se realizan directamente en C, sin intervención de Assembly ni de una biblioteca compartida.

El resultado final se presenta de dos maneras complementarias. En primer lugar, `print_table` muestra una tabla con año, valor original, valor entero y valor ajustado. En segundo lugar, `plot_points_console` genera una representación ASCII donde la longitud de cada barra depende del valor final. Este formato es simple, pero suficiente para evidenciar visualmente la transformación de los datos sin depender de bibliotecas gráficas externas.

0.6.6 Compilación y depuración

La compilación se resuelve mediante un `Makefile` que utiliza `gcc` con las opciones `-O0`, `-g` y `-fno-omit-frame-pointer`. Estas banderas no son accidentales: están elegidas para favorecer la depuración con GDB, preservando información simbólica y una disposición de stack más fácil de inspeccionar.

Además, el `README.md` asociado a esta variante incluye:

- comandos de compilación y ejecución;
- casos de prueba con distintos países y rangos de años;
- ejemplos de inspección del stack antes, durante y después de funciones clave;
- secuencias de uso de GDB sobre `main`, `fetch_https_response`, `extract_http_body`, `parse_world_bank_json`, `convert_points_to_int` e `increment_points`.

Esto convierte a `c_program` en una variante particularmente valiosa para el análisis interno del programa, ya que facilita observar cómo cambian los frames, las variables locales y los

buffers de memoria a medida que avanza el flujo.

0.7 Análisis de Memoria con GDB

Una vez desarrollada la variante `c_program`, se utilizó GDB para observar el estado del stack antes, durante y después de la función `parse_world_bank_json`. Esta función es especialmente relevante porque constituye el punto donde la respuesta HTTP ya fue descargada y el programa comienza a interpretar activamente el contenido JSON para poblar la estructura de datos `IndicatorPoint`.

El objetivo de esta etapa no fue solamente verificar que el programa funcione, sino estudiar cómo cambia la memoria del stack cuando se pasa desde `main` al análisis interno del JSON y luego se retorna nuevamente al llamador. Para ello, se trabajó con comandos como `up`, `down`, `info frame`, `info locals`, `info args`, `print` y `x/32gx $sp`, que permiten inspeccionar tanto la organización lógica de los frames como el contenido crudo de memoria asociado al stack pointer.

0.7.1 Secuencia 1: estado previo al análisis

En la primera captura se observa el estado del programa una vez alcanzado el breakpoint de `parse_world_bank_json`, pero visualizado desde el frame del llamador, es decir, `main`. Para llegar a este punto se usó `up`, con lo cual GDB sube un nivel en la pila de llamadas y deja visible el contexto desde el cual la función de análisis fue invocada.

```
(gdb) up
#1  0x0000555555557a92 in main (argc=2, argv=0x7fffffffd7b8) at main.c:488
488         point_count = parse_world_bank_json(
(gdb) info frame
Stack level 1, frame at 0x7fffffffd6a0:
 rip = 0x555555557a92 in main (main.c:488); saved rip = 0x7ffff742a1ca
 caller of frame at 0x7fffffffd0c0
 source language c.
 Arglist at 0x7fffffffd690, args: argc=2, argv=0x7fffffffd7b8
 Locals at 0x7fffffffd690, Previous frame's sp is 0x7fffffffd6a0
 Saved registers:
  rbp at 0x7fffffffd690, rip at 0x7fffffffd698
(gdb) info locals
country = 0x7fffffffdc06 "Argentina"
start_year = 2011
end_year = 2020
request_path = "/v2/en/country/all/indicator/SI.POV.GINI?format=json&date=2011:2020&page=1&per_page=32500", '\000' <repeats 166 ti
mes>
resolved_country = '\000' <repeats 127 times>
http_response = 0x7ffff7e27010 "HTTP/1.1 200 OK\r\nDate: Wed, 15 Apr 2026 20:29:09 GMT\r\nContent-Type: application/json;charset=u
tf-8\r\nTransfer-Encoding: chunked\r\nConnection: close\r\nServer: cloudflare\r\nCF-Ray: 9ecdb362f8bfc084-EZE\r\nCF-"...
response_body = 0x7ffff7e276b3 "1f8db\r\n[{\"page\":1,\"pages\":1,\"per_page\":32500,\"total\":2660,\"sourceid\":\"2\",\"lastupdat
ed\":\"2026-04-08\"},[{\"indicator\":{\"id\":\"SI.POV.GINI\",\"value\":\"Gini index\"},\"country\":{\"id\":\"ZH\",\"value\":\"Afri
ca Easter"...
response_size = 508708
points = {{year = 0, original_value = 0, converted_value = 0, adjusted_value = 0} <repeats 64 times>}
point_count = 0
(gdb) x/32gx $sp
0x7fffffffd0c0: 0x00007fffffffd7b8      0x00000002f7fc5950
0x7fffffffd0d0: 0x000007dbf7ffdab0      0x00000000000007e4
0x7fffffffd0e0: 0x00007ffff7e27010      0x00007ffff7e276b3
0x7fffffffd0f0: 0x000000000007c324      0x00007fffffffdc06
0x7fffffffd100: 0x0000000000000000      0x0000000000000000
0x7fffffffd110: 0x0000000000000000      0x0000000000000000
0x7fffffffd120: 0x0000000000000000      0x0000000000000000
0x7fffffffd130: 0x0000000000000000      0x0000000000000000
0x7fffffffd140: 0x0000000000000000      0x0000000000000000
0x7fffffffd150: 0x0000000000000000      0x0000000000000000
0x7fffffffd160: 0x0000000000000000      0x0000000000000000
0x7fffffffd170: 0x0000000000000000      0x0000000000000000
0x7fffffffd180: 0x0000000000000000      0x0000000000000000
0x7fffffffd190: 0x0000000000000000      0x0000000000000000
0x7fffffffd1a0: 0x0000000000000000      0x0000000000000000
0x7fffffffd1b0: 0x0000000000000000      0x0000000000000000
(gdb)
```

En este momento, `info frame` indica que el frame activo corresponde a `main`, y `info locals` muestra variables ya preparadas para la llamada, entre ellas:

- `country = "Argentina"`;
- `start_year = 2011`;
- `end_year = 2020`;
- `request_path` ya armado con la URL del indicador;
- `http_response` y `response_body` apuntando al buffer recibido;
- `response_size` con el tamaño de la respuesta descargada;
- `points` todavía inicializado con valores nulos;

- `point_count = 0`.

La observación más importante desde el punto de vista del stack es que el frame de `main` ya contiene todas las direcciones y punteros necesarios para invocar la rutina de análisis, pero aún no presenta resultados procesados. En la volcada de memoria realizada con `x/32gx $sp` predominan punteros, direcciones de retorno y zonas todavía en cero. Esto refleja un estado previo al trabajo de parseo: el stack ya sostiene el contexto de la llamada, pero las estructuras de salida aún no fueron completadas.

La imagen permite precisar aún más esta situación. En la parte superior puede verse que GDB ubica la ejecución en `main.c:488`, exactamente en la línea donde se realiza la asignación:

```
point_count = parse_world_bank_json(...)
```

Esto confirma que todavía no se ingresó al cuerpo efectivo de la función desde la perspectiva del frame actual, sino que el programa está posicionado en el sitio de llamada. También resulta significativo que `info frame` reporte un `saved rip`, es decir, la dirección de retorno que permitirá continuar la ejecución de `main` luego de que la rutina de análisis finalice. En otras palabras, el stack ya contiene la información necesaria para reanudar el flujo del programa, aun antes de que la función llamada produzca resultados.

Otro aspecto visible en la captura es la coexistencia de datos de distinto tipo dentro del mismo frame: por un lado aparecen parámetros simples como `start_year` y `end_year`; por otro, punteros a buffers más complejos, como `http_response` y `response_body`, que remiten a bloques en memoria fuera del frame inmediato. Esta diferencia es importante porque muestra que el stack no contiene necesariamente todos los datos completos, sino principalmente referencias hacia regiones más grandes reservadas en otras áreas de memoria.

La salida de `x/32gx $sp` refuerza esta lectura. Allí se observan valores que corresponden a direcciones válidas del proceso, junto con una secuencia larga de celdas nulas. El patrón es coherente con un estado inicial de trabajo: el frame de `main` ya reservó espacio para variables como `points`, pero ese espacio todavía no fue llenado con información útil. Desde una perspectiva didáctica, esta primera instantánea representa el momento en que la pila está preparada para delegar el control, aunque todavía no contiene los efectos de la computación que vendrá a continuación.

0.7.2 Secuencia 2: estado durante la ejecución de `parse_world_bank_json`

La segunda captura muestra el estado interno una vez que se retorna al frame de `parse_world_bank_json` mediante `down`. Aquí el análisis ya se encuentra ejecutándose dentro de su propio contexto de activación.

```
(gdb) down
#0  parse_world_bank_json (
    json_body=0x7ffff7e276b3 "1f8db\r\n[{\"page\":1,\"pages\":1,\"per_page\":32500,\"total\":2660,\"sourceid\":\"2\",\"lastupdated
\":\"2026-04-08\"},[{\"indicator\":{\"id\":\"SI.POV.GINI\",\"value\":\"Gini index\"},\"country\":{\"id\":\"ZH\",\"value\":\"Africa
 Easter"..., country_name=0x7fffffffdc06 "Argentina", points=0x7fffffffd100, max_points=64, resolved_country=0x7fffffffd500 "",
    resolved_country_size=128) at main.c:283
283     ) {
(gdb) info frame
Stack level 0, frame at 0x7fffffffd0c0:
 rip = 0x555555557042 in parse_world_bank_json (main.c:283); saved rip = 0x555555557a92
 called by frame at 0x7fffffffd6a0
 source language c.
 Arglist at 0x7fffffffd0b0, args:
    json_body=0x7ffff7e276b3 "1f8db\r\n[{\"page\":1,\"pages\":1,\"per_page\":32500,\"total\":2660,\"sourceid\":\"2\",\"lastupdated
\":\"2026-04-08\"},[{\"indicator\":{\"id\":\"SI.POV.GINI\",\"value\":\"Gini index\"},\"country\":{\"id\":\"ZH\",\"value\":\"Africa
 Easter"..., country_name=0x7fffffffdc06 "Argentina", points=0x7fffffffd100, max_points=64, resolved_country=0x7fffffffd500 "",
    resolved_country_size=128
 Locals at 0x7fffffffd0b0, Previous frame's sp is 0x7fffffffd0c0
 Saved registers:
  rbp at 0x7fffffffd0b0, rip at 0x7fffffffd0b8
(gdb) info args
json_body = 0x7ffff7e276b3 "1f8db\r\n[{\"page\":1,\"pages\":1,\"per_page\":32500,\"total\":2660,\"sourceid\":\"2\",\"lastupdated\"
:\"2026-04-08\"},[{\"indicator\":{\"id\":\"SI.POV.GINI\",\"value\":\"Gini index\"},\"country\":{\"id\":\"ZH\",\"value\":\"Africa E
aster"...
country_name = 0x7fffffffdc06 "Argentina"
points = 0x7fffffffd100
max_points = 64
resolved_country = 0x7fffffffd500 ""
resolved_country_size = 128
(gdb) info locals
cursor = 0x313a226c616d6963 <error: Cannot access memory at address 0x313a226c616d6963>
count = 572668450
(gdb) x/32gx $sp
0x7fffffffcf90: 0x0000000000000080      0x00007fffffffd500
0x7fffffffcfa0: 0x0000004074616422      0x00007fffffffd100
0x7fffffffcfb0: 0x00007fffffffdc06      0x00007ffff7e276b3
0x7fffffffcfc0: 0x626f222c22223a22      0x7375746174735f73
0x7fffffffcfd0: 0x6564222c22223a22      0x313a226c616d6963
0x7fffffffcfe0: 0x69646e69227b2c7d      0x7b3a22726f746163
0x7fffffffcff0: 0x4953223a22646922      0x4e49472e564f502e
0x7fffffffd000: 0x756c6176222c2249      0x696e6947223a2265
0x7fffffffd010: 0x7d227865646e6920      0x72746e756f63222c
0x7fffffffd020: 0x226469227b3a2279      0x76222c22575a223a
0x7fffffffd030: 0x5a223a2265756c61      0x2265776261626d69
0x7fffffffd040: 0x746e756f63222c7d      0x6f63336f73697972
0x7fffffffd050: 0x45575a223a226564      0x2265746164222c22
0x7fffffffd060: 0x2c2236313032223a      0x3a2265756c617622
0x7fffffffd070: 0x6e75222c6c6c756e      0x222c22223a227469
0x7fffffffd080: 0x746174735f73626f      0x0000555555556fa8
(gdb)
```

El comando `info frame` confirma que el frame activo ahora es el de `parse_world_bank_json`, y `info args` exhibe los argumentos efectivamente recibidos por la función:

- `json_body`, apuntando al inicio del contenido JSON;

- `country_name`, con el valor `"Argentina"`;
- `points`, apuntando al arreglo donde se guardarán los resultados;
- `max_points = 64`;
- `resolved_country`, apuntando al buffer de salida;
- `resolved_country_size = 128`.

Por su parte, `info locals` deja ver la creación de variables internas propias del análisis, como `cursor`, `count` y buffers auxiliares. Aquí aparece uno de los fenómenos más importantes para la materia: al entrar en una nueva función, el stack crece y se construye un frame adicional que contiene:

- la dirección de retorno al llamador;
- el `rbp` guardado del frame anterior;
- el espacio reservado para argumentos y variables locales;
- copias temporales y punteros auxiliares usados por el parseo.

En la captura, la volcada de `x/32gx $sp` muestra una memoria mucho más activa que en la etapa previa. Ya no predominan únicamente zonas en cero, sino valores que corresponden a direcciones válidas, fragmentos de texto del JSON e información temporal del algoritmo. En otras palabras, el stack deja de ser un simple soporte de invocación y pasa a contener el estado de trabajo concreto de la función. Esta transición es precisamente la evidencia de cómo el stack se modifica dinámicamente al entrar en una rutina con lógica propia.

La captura resulta especialmente interesante porque deja ver el contraste entre argumentos y variables locales. `info args` enumera parámetros que llegan desde `main`, como `json_body`, `country_name` y `points`, mientras que `info locals` expone entidades creadas dentro de `parse_world_bank_json`, entre ellas `cursor` y `count`. Esta distinción permite observar en forma concreta cómo una función combina información heredada del llamador con estado propio reservado en su frame local.

Además, la imagen muestra un detalle valioso: algunos valores locales todavía no se encuentran estabilizados o presentan contenidos transitorios. Por ejemplo, `count` aparece con un valor inconsistente y `cursor` no resulta todavía interpretable de forma correcta. Esto no debe leerse como un error del programa, sino como una consecuencia normal de inspeccionar la función en un momento intermedio de ejecución, antes de que todas las variables hayan sido inicializadas o consolidadas en un punto lógico estable. Desde el punto de vista del análisis del stack, este fenómeno es útil porque revela que la pila no contiene únicamente estado “final”, sino también estados parciales y temporales propios de la ejecución en curso.

La volcada hexadecimal también aporta evidencia sobre el crecimiento efectivo del frame. En comparación con la secuencia anterior, aquí aparecen más direcciones cercanas entre sí y más palabras con contenido no nulo. Esto sugiere que el espacio de stack reservado para la función está siendo utilizado activamente para almacenar referencias, offsets y resultados parciales del parseo. A su vez, el `saved rip` reportado por `info frame` señala que este nuevo frame queda encadenado con el de `main`, preservando la continuidad del control de ejecución. Lo que se ve, por lo tanto, no es solo una colección de variables, sino una evidencia material de la llamada a función y de la estructura jerárquica de la pila.

0.7.3 Secuencia 3: estado posterior al retorno desde la función

La tercera captura corresponde al momento posterior al retorno desde `parse_world_bank_json` hacia `main`. En este punto, la rutina de análisis ya finalizó y el programa vuelve a ejecutarse en el frame llamador.

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffd6a0:
 rip = 0x555555557a92 in main (main.c:488); saved rip = 0x7ffff742a1ca
 source language c.
 Arglist at 0x7fffffffd690, args: argc=2, argv=0x7fffffffd7b8
 Locals at 0x7fffffffd690, Previous frame's sp is 0x7fffffffd6a0
 Saved registers:
  rbp at 0x7fffffffd690, rip at 0x7fffffffd698
(gdb) info locals
country = 0x7fffffffdc06 "Argentina"
start_year = 2011
end_year = 2020
request_path = "/v2/en/country/all/indicator/SI.POV.GINI?format=json&date=2011:2020&page=1&per_page=32500", '\000' <repeats 166 ti
mes>
resolved_country = "Argentina", '\000' <repeats 118 times>
http_response = 0x7ffff7e27010 "HTTP/1.1 200 OK\r\nDate: Wed, 15 Apr 2026 20:29:09 GMT\r\nContent-Type: application/json;charset=u
tf-8\r\nTransfer-Encoding: chunked\r\nConnection: close\r\nServer: cloudflare\r\nCF-Ray: 9ecdb362f8bfc084-EZE\r\nCF-"...
response_body = 0x7ffff7e276b3 "1f8db\r\n[{\"page\":1,\"pages\":1,\"per_page\":32500,\"total\":2660,\"sourceid\":\"2\",\"lastupdat
ed\":\"2026-04-08\"},[{\"indicator\":{\"id\":\"SI.POV.GINI\",\"value\":\"Gini index\"},\"country\":{\"id\":\"ZH\",\"value\":\"Afri
ca Easter"...
response_size = 508708
points = {{year = 2020, original_value = 42.7000008, converted_value = 0, adjusted_value = 0}, {year = 2019,
    original_value = 43.2999992, converted_value = 0, adjusted_value = 0}, {year = 2018, original_value = 41.7000008,
    converted_value = 0, adjusted_value = 0}, {year = 2017, original_value = 41.4000015, converted_value = 0,
    adjusted_value = 0}, {year = 2016, original_value = 42.2999992, converted_value = 0, adjusted_value = 0}, {year = 2014,
    original_value = 41.7999992, converted_value = 0, adjusted_value = 0}, {year = 2013, original_value = 41.0999985,
    converted_value = 0, adjusted_value = 0}, {year = 2012, original_value = 41.4000015, converted_value = 0,
    adjusted_value = 0}, {year = 2011, original_value = 42.7000008, converted_value = 0, adjusted_value = 0}, {year = 0,
    original_value = 0, converted_value = 0, adjusted_value = 0} <repeats 55 times>}
point_count = 0
(gdb) print point_count
$2 = 0
(gdb) print points[0]
$3 = {year = 2020, original_value = 42.7000008, converted_value = 0, adjusted_value = 0}
(gdb) print resolved_country
$4 = "Argentina", '\000' <repeats 118 times>
(gdb) x/32gx $sp
0x7fffffffd0c0: 0x00007fffffffd7b8      0x00000002f7fc5950
0x7fffffffd0d0: 0x000007dbf7ffdab0      0x00000000000007e4
0x7fffffffd0e0: 0x00007ffff7e27010      0x00007ffff7e276b3
0x7fffffffd0f0: 0x000000000007c324      0x00007fffffffdc06
0x7fffffffd100: 0x422acccd000007e4      0x0000000000000000
0x7fffffffd110: 0x422d3333000007e3      0x0000000000000000
0x7fffffffd120: 0x4226cccd000007e2      0x0000000000000000
0x7fffffffd130: 0x4225999a000007e1      0x0000000000000000
0x7fffffffd140: 0x42293333000007e0      0x0000000000000000
0x7fffffffd150: 0x42273333000007de      0x0000000000000000
0x7fffffffd160: 0x42246666000007dd      0x0000000000000000
0x7fffffffd170: 0x4225999a000007dc      0x0000000000000000
0x7fffffffd180: 0x422acccd000007db      0x0000000000000000
0x7fffffffd190: 0x0000000000000000      0x0000000000000000
0x7fffffffd1a0: 0x0000000000000000      0x0000000000000000
0x7fffffffd1b0: 0x0000000000000000      0x0000000000000000
(gdb)
```

Aquí `info frame` vuelve a mostrar `main` como contexto activo, pero con una diferencia fundamental respecto de la primera secuencia: las variables de salida ya fueron modificadas por la función llamada. En particular, se observa que:

- `point_count` deja de valer `0` y pasa a reflejar la cantidad de registros encontrados;
- `resolved_country` contiene efectivamente la cadena `"Argentina"`;
- el arreglo `points` ya no está vacío, sino que sus primeras posiciones poseen valores concretos, por ejemplo `year = 2020` y `original_value = 42.700008`.

Desde la perspectiva del stack, lo central es que el frame local de `parse_world_bank_json` desaparece al completarse la función. Sus variables internas dejan de existir como contexto activo y el stack vuelve a quedar dominado por el frame de `main`. Sin embargo, los efectos

de la función permanecen, porque los datos relevantes fueron escritos a través de punteros en regiones de memoria accesibles por el llamador.

La comparación entre la primera y la tercera imagen evidencia con claridad este comportamiento: el stack en `main` recupera su forma general, pero las zonas referenciadas por `points` y `resolved_country` ya contienen información útil. Es decir, el frame de la función analizada se destruye, pero los cambios producidos sobre memoria compartida o pasada por referencia permanecen observables una vez que el control retorna.

La captura permite verificar esto con varios indicadores concretos. En primer lugar, `info frame` vuelve a ubicar la ejecución en `main.c:488`, igual que en la primera secuencia, pero ahora el contexto ya no es preparatorio sino posterior al retorno. En segundo lugar, `print point_count` muestra el valor `9`, indicando que la función logró identificar y almacenar nueve registros válidos para el país consultado. Esto significa que la llamada no solo retornó correctamente, sino que además dejó un resultado interpretable en el llamador.

La inspección de `points[0]` y del arreglo completo muestra un cambio mucho más profundo que el simple retorno del control. Donde antes había estructuras vacías, ahora aparecen elementos concretos con `year`, `original_value`, `converted_value` y `adjusted_value`. En la imagen se observa, por ejemplo, que los valores originales de varios años ya fueron copiados al arreglo. Aunque las etapas de conversión y ajuste todavía no se ejecutaron, el parseo ya produjo su efecto central: transformar el JSON en una estructura interna ordenada y usable por el resto del programa.

También es relevante notar que la volcada de `x/32gx $sp` vuelve a parecerse a la del caso inicial, pero no de manera idéntica. El frame de `parse_world_bank_json` ya no está presente, por lo que desaparecen sus variables temporales; sin embargo, el contenido asociado a estructuras persistentes ya no está en cero. Este contraste entre "frame recuperado" y "datos modificados" es una de las mejores evidencias del comportamiento real de la pila: la memoria local de la función se destruye, pero los cambios efectuados sobre objetos compartidos sobreviven al retorno. En términos de Sistemas de Computación, esta imagen ilustra con claridad la diferencia entre duración del frame y duración de los datos a los que se accede mediante punteros.

0.7.4 Interpretación general de la variación del stack

Las tres secuencias permiten extraer una lectura precisa sobre el comportamiento de la pila de ejecución:

1. Antes de entrar en la función, el stack de `main` contiene argumentos, punteros y buffers preparados, pero todavía no hay resultados del análisis.
2. Durante la ejecución de `parse_world_bank_json`, se crea un nuevo frame que agrega variables locales, direcciones temporales y estado intermedio del algoritmo.
3. Después del retorno, ese frame desaparece, pero los datos escritos por la función quedan reflejados en las estructuras visibles desde `main`.

Desde el punto de vista de Sistemas de Computación, esta observación es particularmente valiosa porque muestra de manera concreta varios conceptos de la materia:

- la creación y destrucción de stack frames;
- el uso de `rbp` y `rsp` para organizar la pila;
- la distinción entre variables locales y datos persistentes escritos por referencia;
- el papel de la dirección de retorno y de los registros guardados;
- la relación entre la convención de llamada y la disposición real de la memoria durante la ejecución.

En síntesis, la depuración realizada con GDB confirma no solo el funcionamiento correcto del programa, sino también la forma en que la memoria del stack cambia al atravesar una llamada a función con procesamiento interno significativo. La combinación entre comandos de inspección textual y soporte visual mediante las capturas permite transformar la ejecución del programa en una evidencia clara del comportamiento dinámico de la pila.

0.8 Conclusión

El desarrollo realizado a lo largo de este trabajo permite extraer una conclusión central: cuando una misma aplicación distribuye su lógica entre distintos niveles de abstracción, el aspecto verdaderamente crítico no es solo qué hace cada componente, sino de qué manera se comunican entre sí respetando una convención de llamada común. En otras palabras, el problema principal no se reduce a consultar una API, transformar números o mostrar resultados, sino a garantizar que los argumentos, direcciones, tipos y valores de retorno circulen correctamente entre contextos de ejecución diferentes.

Las distintas variantes del proyecto permiten observar esta cuestión desde perspectivas complementarias. En `without_docker`, el intercambio entre Python y la biblioteca nativa evidencia la necesidad de definir con precisión cómo se representan los datos al pasar desde estructuras de alto nivel hacia arreglos contiguos de memoria. En `docker_test`, la relación entre Python, C y Assembly expone con mayor claridad la importancia de respetar la ABI esperada por cada capa, tanto en el orden de los parámetros como en el uso de registros, el tratamiento de punteros y la interpretación del valor de retorno. Finalmente, en `c_program`, el análisis con GDB hace visible la materialización concreta de estas convenciones sobre la pila de ejecución: frames que se crean y destruyen, registros preservados, direcciones de retorno y variables locales que existen solo dentro del alcance de una llamada.

Desde la perspectiva de Sistemas de Computación, este recorrido demuestra que las convenciones de llamada no son un detalle secundario de implementación, sino una condición estructural para que el programa funcione correctamente. Si una capa interpreta mal el tamaño de un dato, pasa un puntero inválido, altera un registro que debía preservarse o retorna un valor con un formato distinto al esperado, la comunicación completa entre componentes queda comprometida. Por eso, comprender cómo se arma un frame, dónde quedan ubicados los argumentos, qué información se conserva al entrar y salir de una función, y cómo persisten los cambios realizados por referencia resulta tan importante como entender la lógica funcional del programa mismo.

En este sentido, el trabajo realizado no solo valida el flujo del índice de Gini como caso de estudio, sino que también confirma en la práctica varios conceptos fundamentales de la materia: la organización de la pila, el paso de parámetros entre funciones, la relación entre lenguaje de alto nivel y código nativo, y la necesidad de una disciplina rigurosa en la interfaz entre módulos. La observación del comportamiento antes, durante y después de llamadas clave permite ver que la ejecución de un programa no es una secuencia abstracta de instrucciones aisladas, sino una coordinación precisa entre contextos de memoria y reglas de invocación que deben cumplirse de manera consistente.

Por lo tanto, la principal enseñanza del trabajo es que las convenciones de llamada constituyen uno de los mecanismos fundamentales que hacen posible la integración entre niveles de software. Gracias a ellas, un programa escrito en Python puede delegar tareas en C, una rutina en C puede apoyarse en Assembly, y un mismo flujo puede mantener coherencia semántica y estabilidad en memoria a través de todas sus etapas. Analizar este proceso en un caso real permitió trasladar conceptos teóricos de la arquitectura y de la ejecución de programas a una evidencia concreta, observable y verificable sobre la máquina.